New Horizons
Centro de Capacitación

Práctica: Interceptores HTTP (RFC 7807 y Loading) y Reactividad con Signals

Paso a paso — Construir interceptores funcionales para Spinner de carga global y mapeo automático de errores ProblemDetails

Curso	Técnico de Desarrollo Full Stack con C# y Angular
Sesión	Sesión 11 — Consumo de la API REST de C# con HttpClient y RxJS
Docente	Edwin Barrientos
Duración estimada	60–90 minutos
Nivel	Intermedio
Entorno	.NET 8 SDK · Visual Studio 2022 Community o VS Code · PostgreSQL o SQL Server Express

1. Objetivos

- Crear LoadingService basado en Signals para controlar la visibilidad del indicador de progreso.
- Implementar loadingInterceptor mediante finalize() de RxJS para conteo de peticiones activas.
- Construir errorInterceptor para capturar respuestas HTTP 400, 404, 422 y 500 del backend de C#.
- Mapear la respuesta de error ProblemDetails (RFC 7807) y desplegar mensajes amigables con MatSnackBar.
- Integrar toSignal() de @angular/core/rxjs-interop para reactividad sin suscripciones manuales.

2. Desarrollo paso a paso

Paso 1 — Crear LoadingService con Signals

En src/app/services/loading.service.ts controlamos el estado del spinner global:

src/app/services/loading.service.ts

```
import { Injectable, signal, computed } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class LoadingService {
  private peticionesActivas = signal<number>(0);
  estaCargando = computed(() => this.peticionesActivas() > 0);

  mostrar(): void {
    this.peticionesActivas.update(count => count + 1);
  }

  ocultar(): void {
    this.peticionesActivas.update(count => Math.max(0, count - 1));
  }
}
```

Paso 2 — Implementar loadingInterceptor funcional

Creamos el interceptor en src/app/interceptors/loading.interceptor.ts:

src/app/interceptors/loading.interceptor.ts

```
import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { finalize } from 'rxjs';
import { LoadingService } from '../services/loading.service';

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loader = inject(LoadingService);
  loader.mostrar();
  return next(req).pipe(finalize(() => loader.ocultar()));
};
```

Paso 3 — Implementar errorInterceptor para ProblemDetails (RFC 7807)

Capturamos las excepciones producidas en el ExceptionHandlingMiddleware de C# en src/app/interceptors/error.interceptor.ts:

src/app/interceptors/error.interceptor.ts

```
import { HttpInterceptorFn, HttpErrorResponse } from '@angular/common/http';
import { inject } from '@angular/core';
import { catchError, throwError } from 'rxjs';
import { MatSnackBar } from '@angular/material/snack-bar';

export interface ProblemDetails {
  type?: string;
  title?: string;
  status?: number;
  detail?: string;
  errors?: Record<string, string[]>;
}

export const errorInterceptor: HttpInterceptorFn = (req, next) => {
  const snackBar = inject(MatSnackBar);

  return next(req).pipe(
    catchError((error: HttpErrorResponse) => {
      let mensajeError = 'Ha ocurrido un error inesperado en el servidor.';

      if (error.error && typeof error.error === 'object') {
        const problem = error.error as ProblemDetails;
        if (problem.detail) {
          mensajeError = problem.detail;
        } else if (problem.title) {
          mensajeError = problem.title;
        } else if (problem.errors) {
          mensajeError = Object.values(problem.errors).flat().join(' | ');
        }
      } else if (error.status === 0) {
        mensajeError = 'No fue posible conectar con el backend C# en localhost:5000.';
      }

      snackBar.open(mensajeError, 'Cerrar', {
        duration: 5000,
        horizontalPosition: 'right',
        verticalPosition: 'top'
      });

      return throwError(() => error);
    })
  );
};
```

Paso 4 — Integrar la barra de carga en el Shell

Actualizamos los tres archivos de ShellComponent (shell.component.ts, shell.component.html y shell.component.css) para mostrar mat-progress-bar cuando hay peticiones HTTP activas:

src/app/layout/shell/shell.component.ts

```
import { Component, signal, inject } from '@angular/core';
import { CommonModule } from '@angular/common';
import { RouterOutlet, RouterLink, RouterLinkActive } from '@angular/router';
import { MatToolbarModule } from '@angular/material/toolbar';
import { MatSidenavModule } from '@angular/material/sidenav';
import { MatListModule } from '@angular/material/list';
import { MatIconModule } from '@angular/material/icon';
import { MatButtonModule } from '@angular/material/button';
import { MatProgressBarModule } from '@angular/material/progress-bar';
import { LoadingService } from '../../services/loading.service';

@Component({
  selector: 'app-shell',
  standalone: true,
  imports: [
    CommonModule, RouterOutlet, RouterLink, RouterLinkActive,
    MatToolbarModule, MatSidenavModule, MatListModule, MatIconModule, MatButtonModule,
MatProgressBarModule
  ],
  templateUrl: './shell.component.html',
  styleUrl: './shell.component.css'
})
export class ShellComponent {
  loader = inject(LoadingService);
  sidenavAbierto = signal<boolean>(true);
}
```

src/app/layout/shell/shell.component.html

```
<mat-toolbar color="primary" class="nh-toolbar">
  <button mat-icon-button (click)="sidenavAbierto.set(!sidenavAbierto())">
    <mat-icon>menu</mat-icon>
  </button>
  <span class="nh-logo-title">
    <mat-icon class="logo-icon">account_balance_wallet</mat-icon>
    SolicitudesApp — Sistema de Reembolsos y Compras
  </span>
  <span class="spacer"></span>
  <div class="user-badge">
    <mat-icon>account_circle</mat-icon>
    <span>Juan Pérez (Solicitante)</span>
  </div>
</mat-toolbar>
<mat-progress-bar *ngIf="loader.estaCargando()" mode="indeterminate" color="accent"
class="global-loader"></mat-progress-bar>

<mat-sidenav-container class="nh-sidenav-container">
  <mat-sidenav [opened]="sidenavAbierto()" mode="side" class="nh-sidenav">
    <mat-nav-list>
      <a mat-list-item routerLink="/dashboard" routerLinkActive="nh-item-active">
        <mat-icon matListItemIcon>dashboard</mat-icon>
        <span matListItemTitle>Dashboard</span>
      </a>
      <a mat-list-item routerLink="/solicitudes" routerLinkActive="nh-item-active">
        <mat-icon matListItemIcon>receipt_long</mat-icon>
        <span matListItemTitle>Bandeja Solicitudes</span>
      </a>
      <a mat-list-item routerLink="/solicitudes/nueva" routerLinkActive="nh-item-active">
        <mat-icon matListItemIcon>add_circle</mat-icon>
        <span matListItemTitle>Nueva Solicitud</span>
      </a>
    </mat-nav-list>
  </mat-sidenav>

  <mat-sidenav-content class="nh-content">
    <router-outlet></router-outlet>
  </mat-sidenav-content>
</mat-sidenav-container>
```

src/app/layout/shell/shell.component.css

```
.nh-toolbar { background: linear-gradient(90deg, #024b98 0%, #002d62 100%); color: white; }
.nh-logo-title { display: flex; align-items: center; gap: 8px; font-weight: 600; font-size:
1.05rem; }
.spacer { flex: 1 1 auto; }
.user-badge { display: flex; align-items: center; gap: 6px; font-size: 0.9rem; }
.global-loader { position: absolute; top: 64px; left: 0; right: 0; z-index: 999; }
.nh-sidenav-container { height: calc(100vh - 64px); }
.nh-sidenav { width: 250px; background: #ffffff; border-right: 1px solid #e2e8f0; }
.nh-item-active { background: #eff6ff !important; color: #024b98 !important; font-weight: 600;
border-left: 4px solid #f57d30; }
.nh-content { padding: 24px; background: #f8fafc; overflow-y: auto; }
```

3. Retos adicionales

1. Implementa una cola de reintentos automáticos con retry(2) en peticiones GET idempotentes.
2. Personaliza los colores del MatSnackBar mostrando verde para confirmaciones y rojo para errores.
3. Convierte el resultado de obtenerPorId() a un Signal usando toSignal() en el componente de detalle.
4. Registra las peticiones lentas (más de 2 segundos) en consola con un interceptor de métricas.

4. Entregables

- Barra de progreso visible durante cada llamada HTTP al backend de C#.
- Mapeo automático de errores RFC 7807 (FluentValidation y ReglaDominioException) hacia MatSnackBar.
- Manejo elegante ante caídas de la API (HTTP 0 / CORS) sin excepciones no controladas.
- Código libre de memory leaks gracias a interceptores funcionales y Signals.